# ARRAY

CCE103

# Objectives:

- Define what is array
- Declare and initialized an array
- Use subscript with an array
- Declare and array of objects
- Pass arrays to and return arrays from methods
- Use the Arrays class
- Use the ArrayList class
- Use two-dimensional and multidimensional array

# Array

- is an object which contains elements of a similar data type.
- The elements of an **array** are stored in a contiguous memory location.
- It is a data structure where we store similar elements. We can store only a fixed set of elements in a **Java array**.

# Array (Declare and Initialized)

- General form of declaration (One-Dimensional Array)

  *datatype [] arrayName;  // Line 1*

  - Where datatype is the element type.

- The general syntax to instantiate an array object is:

  *arrayName = new dataType[intExp];  // Line 2*

  - Where intExp is any expression that evaluates to a positive integer.

- You can combine the statement in Lines 1 and 2 into one statement:

  dataTypr[] arrayName = new datatype[intExp];  // Line 3

- The general form  (syntax) used to access an array element is:

  *arrayName[indexExp];*

# Array (Declare and Initialized)

- intExp = number of components in array >=0
- 0 <= indexExp <= intExp

- In Java, [] is an operator called the ***array subscripting operator.***

# Array

- Array num:

```
int[] num = new int[5];
```

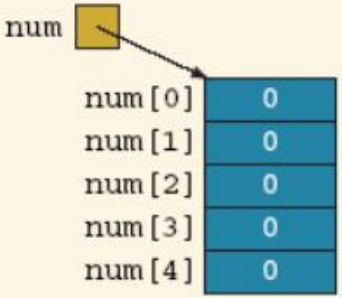


FIGURE 9-1 Array num

# Array

- Array num:

```
int[] num = new int[5];
```

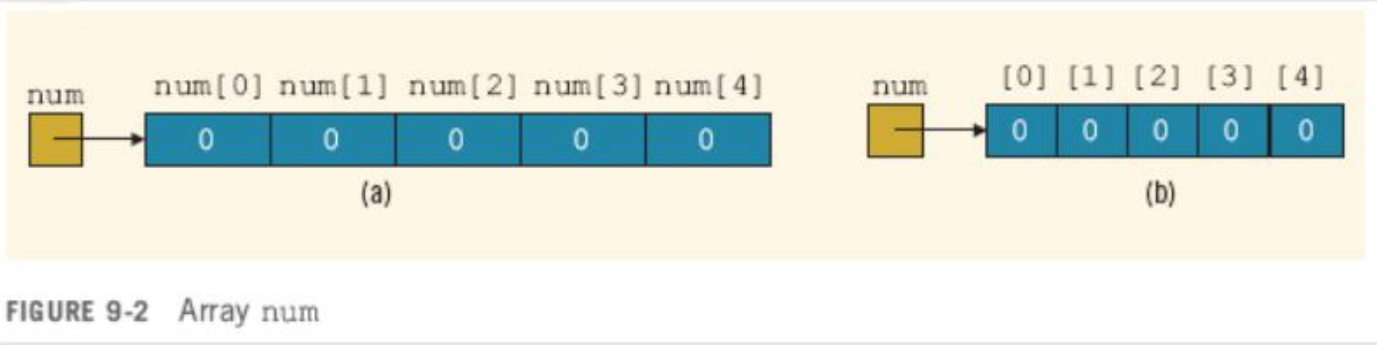


FIGURE 9-2 Array num

# Array

- Array List

```
int[] list = new int[10];
```

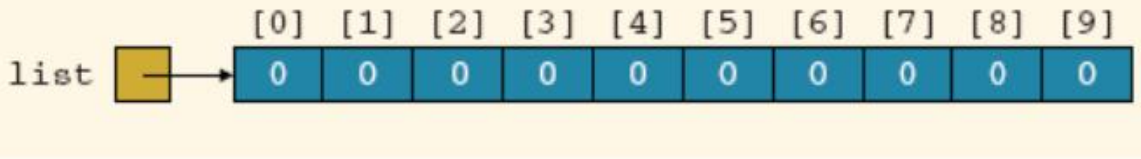


FIGURE 9-3 Array list

# Array

list [ 5 ] = 34;

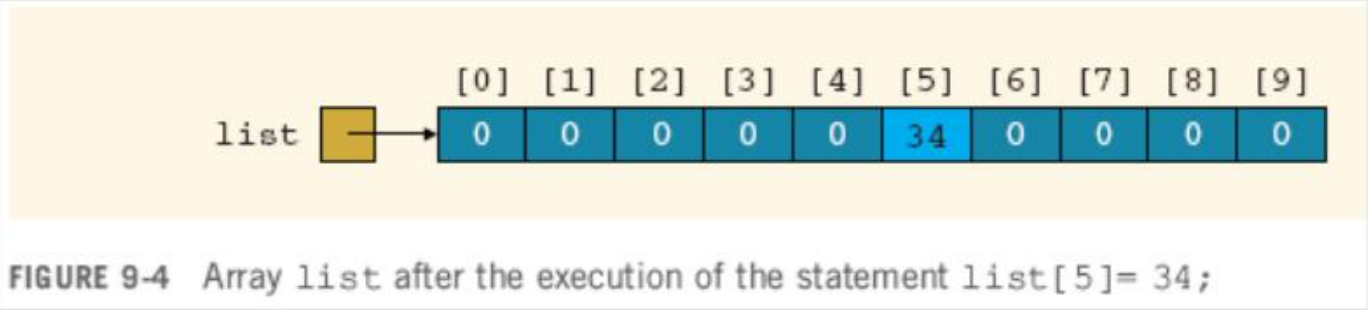


**FIGURE 9-4** Array `list` after the execution of the statement `list[5]= 34;`

# Array

- Array List (Cont.)

  list[3] = 10;

  list[6] = 35;

  list[5] = list[3] + list[6];

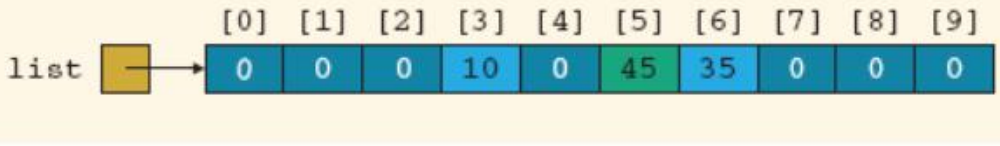


**FIGURE 9-5** Array `list` after the execution of the statements `list[3]= 10;`, `list[6]= 35;`, and `list[5] = list[3] + list[6];`

# Array

**EXAMPLE 9-2**

You can also declare arrays as follows:

```
final int ARRAY_SIZE = 10;
int[] list = new int[ARRAY_SIZE];
```

# Array

## Specifying Array Size during Program Execution

```
int arraySize;                                                  //Line 1

System.out.print("Enter the size of the array: ");  //Line 2
arraySize = console.nextInt();                                  //Line 3
System.out.println();                                           //Line 4

int[] list = new int[arraySize];                                //Line 5
```

# Array

## Array Initialization during Declaration

```
double[] sales = {12.25, 32.50, 16.90, 23, 45.68};
```

- The **initializer list** contains values, **called initial values**, that are placed between braces and separated by commas
- `sales[0]= 12.25`, `sales[1]= 32.50`, `sales[2]= 16.90`, `sales[3]= 23.00`, and `sales[4]= 45.68`

# Array

- Associated with each array that has been instantiated, there is a `public` (`final`) instance variable `length`
- The variable `length` contains the size of the array
- The variable `length` can be directly accessed in a program using the array name and the dot operator

```
int[] list = {10, 20, 30, 40, 50, 60};
```

- This statement creates the array list of six components and initializes the components using the values given
  - *Here `list.length` is 6*

# Array

```
int[] numList = new int[10];
```

- This statement creates the array `numList` of 10 components and initializes each component to 0
- The value of `numList.length` is `10`

```
numList[0] = 5;
numList[1] = 10;
numList[2] = 15;
numList[3] = 20;
```

- These statements store `5`, `10`, `15`, and `20`, respectively, in the first four components of `numList`
- You can store the number of filled elements, that is, the actual number of elements, in the array in a variable, say `numOfElement`
- It is a common practice for a program to keep track of the number of filled elements in an array

# Array

- Loops used to step through elements in array and perform operations

```
int[] list = new int[100];
int i;

for (i = 0; i < list.length; i++)
   //process list[i], the (i + 1)th
   //element of list

for (i = 0; i < list.length; i++)
    list[i] = console.nextInt();

for (i = 0; i < list.length; i++)
    System.out.print(list[i] + " ");
```